

Apostila do Laboratório de Linguagens de Programação

Módulo C++: Programação Orientada a Objetos

Pedro Henrique Ribeiro Dias

7 de setembro de 2026

Sumário

1	Relatório 6: Programação Orientada a Objetos em C++	2
1.1	Fundamentos Teóricos e Pilares da POO	2
1.2	Documentação Sintática das Estruturas de POO	2
1.2.1	Declaração de Classe e Modificadores de Acesso	2
1.2.2	Construtores e Lista de Inicialização	3
1.2.3	Herança Pública	3
1.2.4	Polimorfismo Dinâmico: <code>virtual</code> e <code>override</code>	3
1.2.5	Destrutor Virtual (<code>virtual ~Destrutor()</code>)	3
1.2.6	Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória	3
1.3	Código de Referência Completo	4
1.4	Exercícios Práticos do Módulo	5

Capítulo 1

Relatório 6: Programação Orientada a Objetos em C++

1.1 Fundamentos Teóricos e Pilares da POO

A Programação Orientada a Objetos (POO) é um paradigma de programação fundamentado na estruturação do código em torno de unidades chamadas **objetos**, que combinam estado (atributos) e comportamento (métodos). A POO se sustenta em quatro pilares fundamentais:

- **Abstração:** Mapeamento de entidades do mundo real para o código, isolando apenas as características e comportamentos relevantes para o contexto do sistema.
- **Encapsulamento:** Proteção e isolamento do estado interno de um objeto, controlando o acesso e a modificação de dados por meio de níveis de visibilidade.
- **Herança:** Mecanismo que permite que uma classe reutilize e expanda a estrutura e o comportamento de uma classe ancestral (base).
- **Polimorfismo:** Capacidade de objetos de diferentes classes derivadas responderem à mesma mensagem (chamada de método) de maneiras distintas.

1.2 Documentação Sintática das Estruturas de POO

Esta seção detalha todas as palavras-chave e construções sintáticas utilizadas na implementação de POO em C++.

1.2.1 Declaração de Classe e Modificadores de Acesso

Uma classe é declarada com a palavra-chave `class`. O corpo da classe define os níveis de acesso aos seus membros:

```
1 class SerVivo {  
2     protected:  
3         std::string nome; // Acess vel pela classe e por classes derivadas  
4         (filhas)  
5     public:  
6         // Acess vel de qualquer ponto do programa  
7 };
```

- **private** (Padrão): Membros são acessíveis apenas pelos métodos da própria classe.

- **protected**: Membros são inacessíveis externamente, mas acessíveis dentro da própria classe e por suas subclasses derivadas.
- **public**: Membros podem ser acessados diretamente por qualquer objeto ou função do sistema.

1.2.2 Construtores e Lista de Inicialização

O construtor possui o mesmo nome da classe e é executado automaticamente no momento da instanciação. A sintaxe de **lista de inicialização de membros** (*Member Initializer List*) atribui valores aos membros antes da execução do corpo do construtor:

```
1 // Sintaxe: Construtor(parametros) : atributo(parametro) {}
2 Servivo(string n) : nome(n) {}
```

1.2.3 Herança Pública

A herança estabelece uma relação do tipo "É UM". Para herdar publicamente de uma classe base:

```
1 class Humano : public Servivo {
2 public:
3     // Repassa o parametro 'n' para o construtor da classe base Servivo
4     Humano(string n) : Servivo(n) {}
5 };
```

1.2.4 Polimorfismo Dinâmico: virtual e override

- **virtual** (na classe base): Declara um método que pode ser sobrescrito nas classes derivadas. Habilita o *dynamic dispatch* (ligação tardia em tempo de execução).
- **override** (na classe derivada): Garante em tempo de compilação que o método está sobrescrevendo exatamente uma função virtual da classe base.

```
1 // Na Classe Base:
2 virtual void apresentar() { ... }
3
4 // Na Classe Derivada:
5 void apresentar() override { ... }
```

1.2.5 Destrutor Virtual (virtual ~Destrutor())

Sempre que uma classe possui métodos virtuais e é manipulada por meio de ponteiros da classe base, seu destrutor **deve ser virtual**. Isso garante que o destrutor correto da classe derivada seja invocado na desalocação de memória.

```
1 virtual ~Servivo() {}
```

1.2.6 Ponteiros de Classe Base e Gerenciamento Dinâmico de Memória

O polimorfismo dinâmico exige o uso de ponteiros ou referências da classe base armazenando o endereço de instâncias das classes filhas alocadas dinamicamente via operador **new**.

```

1 // Armazenamento genérico via ponteiro da classe base
2 vector<Servivo*> seres;
3
4 // Instancia o dinâmico
5 seres.push_back(new Humano("Arthur"));
6
7 // Chamada polimórfica via ponteiro (operador ->)
8 for (Servivo* s : seres) {
9     s->apresentar();
10 }
11
12 // Libera a obrigatoria da memória alocada dinamicamente
13 for (Servivo* s : seres) {
14     delete s;
15 }

```

1.3 Código de Referência Completo

Abaixo está a implementação de referência utilizada para exemplificação do paradigma POO em C++:

```

1 #include <iostream>
2 #include <string>
3 #include <vector>
4 using namespace std;
5
6 // CLASSE BASE (Abstrato)
7 class Servivo {
8 protected:
9     string nome; // Encapsulamento protegido
10
11 public:
12     // Construtor com Lista de Inicialização
13     Servivo(string n) : nome(n) {}
14
15     // Método Virtual para Polimorfismo
16     virtual void apresentar() {
17         cout << "Eu sou um ser vivo chamado " << nome << "." << endl;
18     }
19
20     // Destrutor Virtual
21     virtual ~Servivo() {}
22 };
23
24 // HERANÇA: Humano herda de Servivo
25 class Humano : public Servivo {
26 public:
27     Humano(string n) : Servivo(n) {}
28
29     // SOBRESCRITA (Override)
30     void apresentar() override {
31         cout << "Olá, eu sou o humano " << nome << ", mestre das
32         civilizações." << endl;
33     }
34 };

```

```

35 class Elfo : public Servivo {
36 public:
37     Elfo(string n) : Servivo(n) {}
38
39     void apresentar() override {
40         cout << "Sauda es, eu sou o elfo " << nome << ", guardi o
das florestas." << endl;
41     }
42 };
43
44 class Fada : public Servivo {
45 public:
46     Fada(string n) : Servivo(n) {}
47
48     void apresentar() override {
49         cout << "Encantado! Eu sou a fada " << nome << ", portadora da
magia." << endl;
50     }
51 };
52
53 int main() {
54     // Vetor de ponteiros para a classe base (Polimorfismo)
55     vector<Servivo*> seres;
56
57     // Aloca o din mica de mem ria
58     seres.push_back(new Humano("Arthur"));
59     seres.push_back(new Elfo("Legolas"));
60     seres.push_back(new Fada("Luna"));
61     seres.push_back(new Servivo("Pedro"));
62
63     cout << "=== Apresenta es no mundo de fantasia ===" << endl;
64
65     // Execu o polim rfica
66     for (Servivo* s : seres) {
67         s->apresentar();
68     }
69
70     // Desaloca o de mem ria (Good Practices)
71     for (Servivo* s : seres) {
72         delete s;
73     }
74
75     return 0;
76 }

```

Listing 1.1: Exemplo de Aplicação de POO em C++

1.4 Exercícios Práticos do Módulo

Exercício 1: Abstração e Encapsulamento (Sistema de Combate de Robôs)

Crie uma classe `Robo` contendo os atributos `modelo` (string), `versao` (int), `potencialLaser` (float) e `integridade` (int). Implemente o método `disparar(Robo &alvo)` que reduz a integridade do alvo com base na potência do laser do atacante. Instancie dois robôs e simule um confronto.

Exercício 2: Herança e Modificadores de Acesso (Persona RPG)

Implemente a classe base **Pessoa** com atributos privados **nome** e **idade**. Crie as classes derivadas **Protagonista** (com o atributo **nivel**) e **Personagem** (com o atributo **rank**). Demonstre o acesso correto aos membros através dos construtores da classe pai.

Exercício 3: Polimorfismo Dinâmico (Membros do Inatel)

Crie a classe base **MembroInatel** com o método virtual **identificar()**. Derive as classes **Coordenador** (atributo **departamento**) e **Pesquisador** (atributo **laboratorio**). Utilize ponteiros da classe base para chamar o método **identificar()** de forma polimórfica.

Exercício 4: Polimorfismo em Coleções (Conselho das Terras Ancestrais)

Crie a classe base **MembroConselho** e as derivadas **Anao**, **Orc** e **Draconato**. Armazene instâncias dessas classes em um `std::vector<MembroConselho*>` e percorra a coleção invocando o método sobrescrito **saudar()**.